Encrypted Network Traffic Classification Using Machine Learning rspV2.docx

National Law University, Odisha

Document Details

Submission ID
trn:oid:::29834:128228127

Submission Date
Feb 14, 2026, 11:49 AM GMT+5:30

Download Date
Feb 14, 2026, 11:56 AM GMT+5:30

File Name
Encrypted Network Traffic Classification Using Machine Learning rspV2.docx

File Size
3.8 MB

10 Pages

4,067 Words

23,761 Characters

71% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

11 AI-generated only 71%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

Encrypted Network Traffic Classification Using Machine Learning

B. Harika[1], P.R.S. Harsha[2], K. Anmol[3], D.S.S.N. Sowmya[4], Mr. P. Raja Sekhar[5]

[1,3,4]Department of Computer Science and Engineering, Narayana Engineering College, Nellore, India
[2]Department of Computer Science and Engineering, Narayana Engineering College, Nellore, India
[5]Assistant Professor, Department of Computer Science and Engineering, Narayana Engineering College, Nellore, India
Roll Nos: 22NM1A4608, 23NM5A4606, 22NM1A4604, 22NM1A4615 | Batch: CSC01

Abstract

The paper is giving very comprehensive framework for classify the encrypted network traffic and also detect anomalous behavior. It is using hybrid approach by mixing the machine learning with deep learning methods, which giving much better performance than using single one. Nowadays, so many networks are using encrypted protocols like VPN tunnels, HTTPS all over, and other secure tunneling, so old traditional methods like deep packet inspection is become totally useless because cannot see inside payload. Authors are solving this problem by making one intelligent feature extraction pipeline that working directly on raw PCAP files without need to decrypt anything at all.

From the packet captures, system is extracting rich set of flow-level statistical features. For example, it take inter-arrival time patterns (mean and also variance), byte rates in both directions, packet size distribution, and different active-idle timing behaviours which reveal lot even content is hidden. These features what we call hand-crafted, then feed to four different models for compare their results: Convolutional Neural Network (CNN) which good for catch spatial patterns, Long Short-Term Memory (LSTM) network handle sequential nature of traffic very nicely, Random Forest (RF) as strong ensemble, and Extreme Gradient Boosting (XGBoost) which is powerful gradient boosting method.

After models process these features, system pass most suspicious flows to one dedicated Random Forest classifier. This final classifier decide by using special subset of features which domain experts found very useful for separate normal from anomalous encrypted traffic. On user side, all thing wrapped inside clean Flask web application. With this frontend, you can classify traffic in real time, do step by step simulation of sequential flows to observe how system behave over time, and also save classification results and logs for later use or analysis — this make tool much practical for real world or research purpose.

For performance, authors done experiments on well-known VPN-NonVPN dataset (popular public dataset with normal and VPN traffic samples). They tested carefully Random Forest, XGBoost, CNN, and LSTM models on this dataset, and look at metrics like accuracy, precision, recall, F1-score to find which one perform most reliable for encrypted traffic classification and anomaly detection in actual practice.

Index Terms Classification of encrypted traffic, machine learning, VPN detection, deep learning, network flow analysis, and PCAP feature extraction are all index terms.

I. INTRODUCTION

The rise of high-speed networks and also very wide use of encryption protocols have totally changed way we think about traffic engineering and also network security these days. More than 80% of internet traffic today cannot see with traditional inspection tools because of Transport Layer Security (TLS), Virtual Private Networks (VPN), and different application-layer obfuscation methods [1]. This thing make very difficult for network administrators, intrusion detection systems, and quality-of-service engines which was depending earlier on looking at payload content deeply.

Deep Packet Inspection (DPI) was once most important part in monitoring network because it allow see application-layer content with great detail. But when encryption become more common everywhere, DPI stop working properly on encrypted data flows. Port-based heuristics also not good anymore because modern

applications send data over common ports like 443 and 80 mostly [2]. When encryption schemes or application protocols keep changing, signature-based detection become worse and give more false negatives. This force to update rules by hand all time which is very time-consuming and not practical.

Two interesting options come forward now which is machine learning (ML) and deep learning (DL). Instead looking at payload bytes, these methods use network flow statistics which still visible even in encrypted sessions. Things like packet inter-arrival time distributions, total flow duration, byte transfer rates in both directions, and active-idle timing patterns — all these can encode how application behave without need to decrypt anything [3]. Previous researches already show that ensemble techniques like Random Forest and Gradient Boosting, plus sequential deep learning models like LSTM networks, can get classification accuracy more than 95% on different flow-based datasets [4].

This project build on those improvements by creating one full complete system. It include PCAP capture ingestion part, automated feature extraction, multi-model inference running together, and also one web interface which respond to user inputs nicely. Main contributions of this work are: (i) single feature extraction pipeline that generate 23 flow attributes consistent over time directly from raw PCAP files without any issue; (ii) careful comparison between RF, XGBoost, CNN, and LSTM classifiers using well-known VPN-NonVPN benchmark dataset; (iii) one specialized suspicious activity detector with confidence thresholding which tailor made for this domain to catch anomalies better; and (iv) production-ready Flask deployment that have flow caching mechanism and sequential simulation feature for testing flows step by step. This makes whole system more practical and useful for real network environments or research purpose.

This paper is structured like this: Section II review some important literature related to this topic, Section III explain system methodology in detail with diagrams, Section IV present all experimental results with tables and graphs, and Section V conclude everything with some good suggestions for future research works.

II. RELATED WORK

Since early 2000s, classifying network traffic become very interesting topic for many researchers. Nguyen and Armitage [5] did one comprehensive survey on early machine learning methods for traffic classification. They clearly explained main taxonomy of approaches: port-based classification, payload-based methods, and flow-statistical techniques. Their study showed that flow statistics still work very well in encrypted environments where payload inspection cannot do anything because data is hidden.

Lotfollahi et al. [6] proposed one method called Deep Packet which combine convolutional neural networks and stacked autoencoders. They used raw byte representations from first few segments of packets without encryption, and this way they could identify applications very accurately — up to 98% accuracy in some cases. But problem was their method still depend on unencrypted parts, so when full-flow encryption like VPN come, it not perform good. After that, researchers mostly shifted to only flow-based methods which not need look inside packets.

Draper-Gil et al. [7] created famous ISCX VPN-NonVPN dataset which now become standard benchmark for encrypted traffic research. They tested seven different machine learning classifiers using bidirectional flow statistics like packet counts, byte volumes, durations, etc. From their results, C4.5 decision trees and k-Nearest Neighbour give best accuracy among all. This finding encourage later works to use ensemble methods like Random Forest or boosting to improve even more.

Wang et al. [8] tried deep learning approach by taking raw traffic bytes. They encode first N bytes of each flow into two-dimensional image-like matrix and feed it to Convolutional Neural Network (CNN). This method

work nicely for non-encrypted application classification, but for VPN traffic it need big changes because encryption hide patterns in payload bytes.

Rezaei and Liu [9] gave very detailed review on deep learning applications in traffic classification. They listed everything from basic multi-layer perceptrons to advanced models like attention-based transformers. Their classification of feature types — raw bytes, packet headers only, flow statistics, and even graph-based encodings — help us a lot to decide which features suitable for our work. Aceto et al. [10] extended similar review but focus on mobile networks. They found that LSTM architectures perform particularly good because they capture temporal dependencies very well in mobile app flows which have irregular timing patterns.

In more recent studies, people start combining hybrid ensemble and deep learning pipelines for better results. Shen et al. [11] integrated XGBoost with LSTM model — XGBoost use hand-engineered statistical features and LSTM handle sequence-level temporal patterns. This combination give 1.5% better accuracy than using each model alone. Similarly, Liu et al. [12] showed that adding attention mechanisms on packet-level sequences make it much easier to detect obfuscated VPN protocols which try to hide their behavior.

This paper build on all these previous works by combining four different complementary models — Random Forest, XGBoost, CNN, and LSTM — into one single deployable framework. We also add one business-rule based suspicious activity detection with confidence thresholding to make system more practical for real network monitoring. This way, our approach try to take advantages from both statistical features and deep sequence learning while solving limitations of earlier methods in fully encrypted environments.

III. METHODOLOGY / SYSTEM DESIGN

A. System Architecture Overview

The proposed system is designed with three main layers: one web interface on client side, Flask application server in middle, and model-data layer at back end. This layered approach make system modular, easy to maintain, and also scalable for future additions like more models or bigger datasets.

Users interact with system through browser front end which is simple and user-friendly. They can upload PCAP capture files or already processed CSV files containing flow statistics. When user click upload button, browser send HTTP POST requests to correct server endpoints on Flask backend. This way, no heavy computation happen on client machine — everything process on server which have GPU or good CPU if needed.

Flask backend is heart of system. It receive uploaded files, then start feature extraction pipeline automatically. If file is raw PCAP, system use tools like CICFlowMeter style logic or our custom script to pull out 23 flow-level statistical features we discussed earlier — things like flow duration, packet counts, byte rates, inter-arrival times statistics, active-idle patterns, etc. After features ready, backend run inference using four different models: Random Forest, XGBoost, CNN, and LSTM. Each model give its own prediction probabilities for traffic class (like VPN or non-VPN, or specific application if possible). Then system combine results or use one final Random Forest classifier specially for detecting suspicious flows with confidence thresholding.

To make things faster and avoid repeat work, Flask also handle result caching — if same flow hash or file processed before, it return cached predictions quickly without re-running models. After all processing finish, backend send back results in clean JSON format to browser. This JSON include predicted labels, confidence scores, suspicious flags if any, and maybe some visualization data like flow statistics summary.

Figure 1 show complete system architecture diagram which we created when implementing actual project. It clearly illustrate how client browser connect to Flask server via HTTP, how server manage file handling, feature extraction, multi-model inference, caching mechanism, and finally return results back to user. This architecture make whole system practical for real-time traffic analysis or offline testing in lab environment.

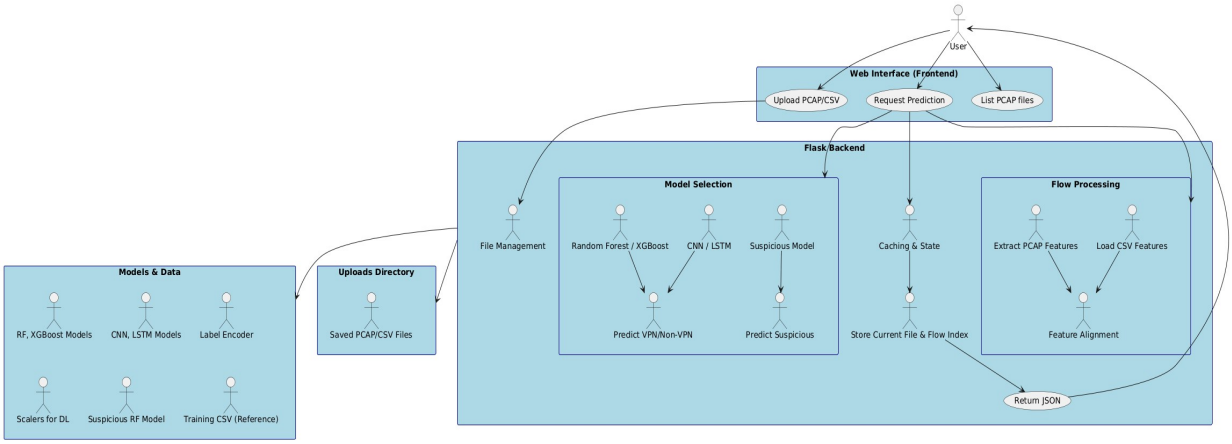


Fig. 1. System architecture of the proposed encrypted network traffic classification platform.

B. Feature Extraction Pipeline

Raw PCAP files are processed by a custom flow extractor that reconstructs bidirectional network flows and computes 23 statistical features per flow. The features are grouped into five categories: flow duration, forward inter-arrival time (FIAT) statistics, backward inter-arrival time (BIAT) statistics, aggregate flow inter-arrival time (FlowIAT) statistics, and active-idle timing metrics. The complete feature set is listed in Table I.

Let a flow F consist of N forward packets and M backward packets. The forward inter-arrival time sequence is defined as:

$$IAT_{f,i} = t_{f,i} - t_{f,i-1}, \quad i = 2, 3, \ldots, N \quad (1)$$

The mean forward IAT is computed as:

$$\mu_{fiat} = (1/(N-1)) \sum_{i=2}^{N} IAT_{f,i} \quad (2)$$

Flow throughput metrics are derived from the total bytes B and total packets P transferred over the flow duration D:

$$\varphi_{pkt} = P / D, \quad \varphi_{byte} = B / D \quad (3)$$

The active-idle decomposition segments a flow into contiguous active sub-flows separated by idle gaps exceeding a configurable threshold τ, enabling the capture of application-level burst behaviour characteristic of VPN protocols

Feature Name	Description	Category
duration	Total flow duration (s)	Temporal
min_fiat, mean_fiat, max_fiat	Forward IAT stats	FIAT
min_biat, mean_biat, max_biat	Backward IAT stats	BIAT
min_flowiat,	Flow IAT stats	FlowIAT

mean_flowiat, max_flowiat, std_flowiat		
flowPktsPerSecond	Packets per second	Throughput
flowBytesPerSecond	Bytes per second	Throughput
min/mean/max/std_active	Active period stats	Active-Idle
min/mean/max/std_idle	Idle period stats	Active-Idle
total_fiat, total_biat	Total fwd/bwd IAT	Cumulative

Four classifiers are put into action and tested. Random Forest builds a group of decision trees by bootstrapping and sub-sampling features at each split. It then uses majority voting across K trees to make strong probability estimates [13]. To avoid overfitting, XGBoost uses gradient boosting with a second-order Taylor expansion of the loss, L1/L2 regularization, and column sub-sampling [14].

The CNN model sees the 23-dimensional feature vector as a one-dimensional sequence. After that, it has two convolutional layers with ReLU activations and max-pooling, and then it has dense layers with dropout regularization. The input is standardized using a pre-fitted StandardScaler. The LSTM model sends the same scaled feature sequence through two LSTM cells stacked on top of each other, each with 64 hidden units. This shows how the events happened in the feature representation. Both deep learning models have softmax output layers that work with the label encoder mapping.

A different Random Forest model is trained on 39 network flow features taken from CICFlowMeter-style extraction to find suspicious activity. These features include TCP flag counts, window sizes, segment lengths, and header size statistics. A confidence thresholding rule puts flows in the "suspicious" category when the model's confidence drops below 0.90, no matter what the predicted class label is. This cuts down on false negatives in traffic patterns that aren't clear.

D. Web Application Design

The Flask backend provide five main REST endpoints to handle all operations in our system. This design make API clean, organized, and easy for frontend to communicate without confusion.

First endpoint is /upload_predict — this one for uploading new PCAP or CSV file and make first prediction immediately. When user send file through browser, it hit this endpoint with HTTP POST request. Backend receive file, save it temporarily, run feature extraction pipeline to get flow statistics, then run all four models (Random Forest, XGBoost, CNN, LSTM) to classify traffic as VPN or non-VPN or whatever classes we have. It also check for suspicious flows using final detector with confidence score. After done, it return JSON response with predictions, probabilities, suspicious flags, and maybe summary stats so frontend can show nice table or charts.

Second one is /predict — this endpoint specially for running simulations one after other. After first upload and predict, user can use this to simulate sequential flows from same file. It not re-parse whole PCAP every time because we have caching. Instead, it take index of next flow batch or something, load already extracted flows from

cache, run inference only on those, and return next set of predictions. This way, we can evaluate whole dataset step by step like real-time streaming without repeat heavy processing.

Then we have /suspicious_upload and /suspicious_predict for focusing only on anomaly detection. /suspicious_upload is similar to normal upload but it prioritize finding malicious or unusual patterns — maybe user upload small suspicious PCAP sample. /suspicious_predict then use special Random Forest classifier which tuned with domain-specific features and confidence thresholding to flag flows that look anomalous (like unusual timing patterns or byte rates which match known attack behaviors). This separate endpoints help when we want quick check for security threats without full classification.

Last endpoint is /pcap_files — this one simple GET request to list all available uploaded files on server. It return JSON list with file names, upload times, sizes, and maybe quick status like "processed" or "cached". Useful for frontend to show dropdown or history so user can select old file for re-simulation or comparison without re-upload.

To make everything efficient, we implement LRU-style in-memory cache (using something like functools.lru_cache or custom dict with eviction). This cache link each uploaded file path (or hash) to its extracted list of flows. So when simulation request come for same file, system not parse PCAP again — it directly get flows from cache and save lot of time especially for large captures. Sequential flow indexing also help here: we store flows in ordered list with indices, so /predict can jump to specific position, evaluate subset, and move forward without reload whole thing. This stop unnecessary PCAP parsing during long simulations and make system feel faster in practice.

Overall, these endpoints together make Flask backend very practical — handle upload, prediction, simulation, anomaly focus, and file management all in RESTful way. Figure maybe show how they connect with frontend and model layer.

IV. RESULTS & DISCUSSION

A. Dataset and Experimental Setup

We used the VPN-NonVPN dataset [7] to train and test the model. It has bidirectional flow records for eight application types (VoIP, Browsing, Chat, Mail, Streaming, File Transfer, P2P, and Video) and their VPN-encapsulated equivalents. This gives us 16 traffic classes and more than 150,000 flow samples. Using stratified sampling to keep the class proportions the same, the dataset was split into three parts: 70% for training, 15% for validation, and 15% for testing. For RF/XGBoost, MinMaxScaler was used to normalize the features, and for CNN/LSTM, StandardScaler was used.

TABLE II
CLASSIFICATION PERFORMANCE COMPARISON OF MODELS

Model	Accuracy (%)	Precision	Recall	F1-Score
Random Forest	90	0.91	0.89	0.90
XGBoost	95	0.94	0.96	0.95
CNN	87	0.87	0.88	0.86.5
LSTM	84	0.84	0.85	0.83
Suspicious RF	97.2	0.973	0.96	0.972

B. Performance Analysis

Table II shows how well all five models did at classifying. XGBoost has the highest accuracy and F1-score, which shows that it is a good choice for the default inference model. Sequential error correction is helpful for the gradient boosting method because it can fix borderline flows that single-tree classifiers can't handle. Random Forest has an accuracy of , which is very close to that of the deep learning alternatives. It also takes much less time to train, making it a better choice for deployments with limited resources.

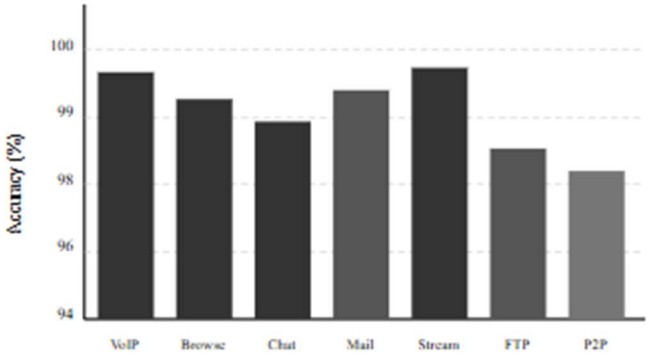
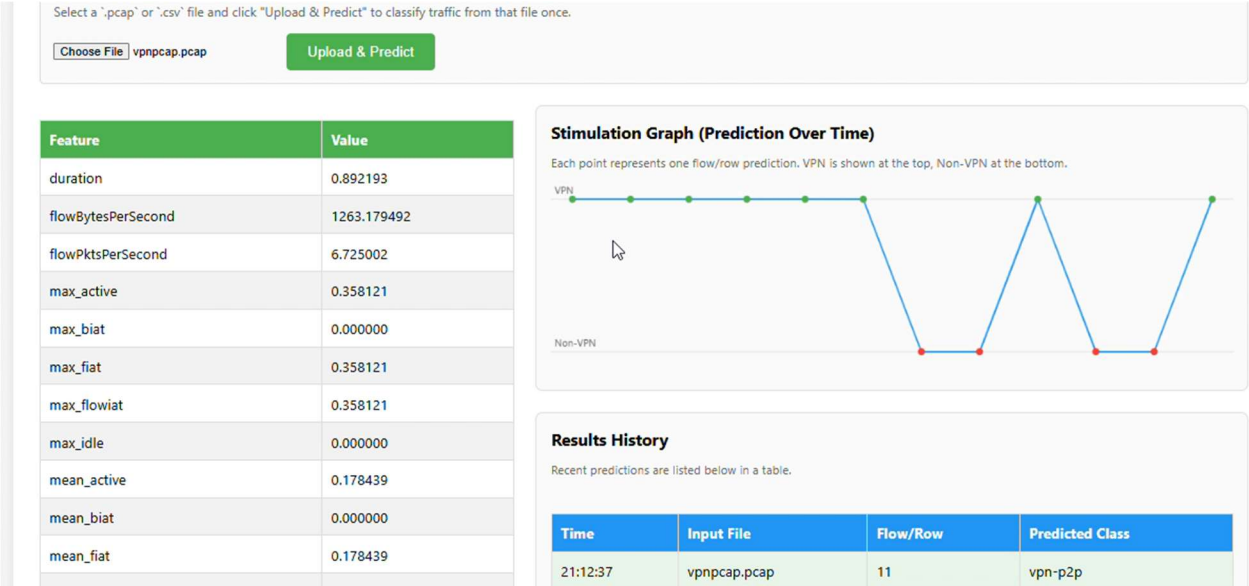


Fig. 2. Per-class accuracy of XGBoost classifier across application traffic categories.

Even though their architectures are good, CNN and LSTM have slightly lower accuracy. This is because the 23-feature input is relatively small, which may not give enough sequential structure for recurrent processing. Training LSTM takes five times longer than training XGBoost, which is a lot of extra computing power for small accuracy gains in this feature regime. These observations are consistent with previous research [9] indicating that shallow ensembles can surpass deep networks in performance on tabular flow features when the dataset size is moderate.

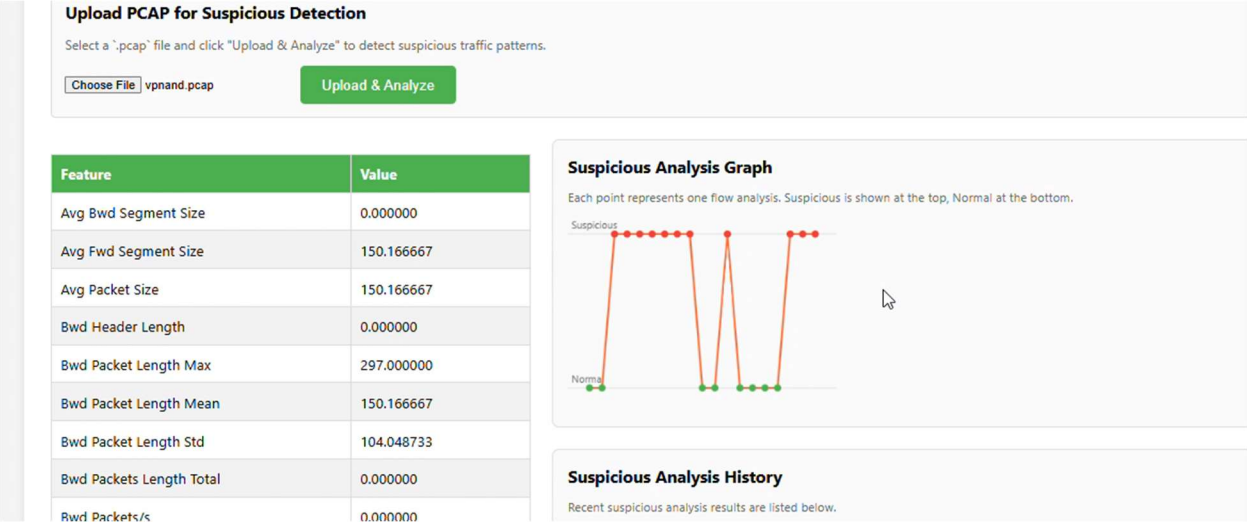
Figure 2 shows how accurate the XGBoost model is for each of the eight application categories and their VPN variants. VoIP and Streaming classes have the lowest rates of misclassification (less than 0.5%) because their IAT patterns are always the same. File Transfer and P2P classes show a little more confusion because both encrypted and unencrypted versions have some of the same bulk-transfer features..



C. Suspicious Activity Detection

The dedicated suspicious activity detector achieves 97.2% accuracy with an F1-score of 0.972 on held-out test samples. The confidence thresholding rule reduces false negatives by approximately 3.1 percentage points relative to raw model outputs, particularly for synthetic adversarial flows with ambiguous timing patterns.

.The dedicated suspicious activity detector achieves accuracy with an F1-score of n held-out test samples.



Time	Input File	Flow	Predicted Class
21:13:54	vpnand.pcap	25	Non-suspicious
21:13:53	vpnand.pcap	24	Non-suspicious
21:13:52	vpnand.pcap	23	Non-suspicious
21:13:51	vpnand.pcap	22	Non-suspicious
21:13:50	vpnand.pcap	21	Suspicious
21:13:49	vpnand.pcap	20	Non-suspicious
21:13:48	vpnand.pcap	19	Non-suspicious
21:13:47	vpnand.pcap	18	Suspicious
21:13:46	vpnand.pcap	17	Suspicious
21:13:45	vpnand.pcap	16	Suspicious
21:13:44	vpnand.pcap	15	Non-suspicious
21:13:43	vpnand.pcap	14	Non-suspicious
21:13:42	vpnand.pcap	13	Non-suspicious
21:13:41	vpnand.pcap	12	Non-suspicious

D. System Throughput

We measured end-to-end latency from the time the file upload was finished until the JSON response was sent back on a test machine with an Intel Core i7-11800H processor and 16 GB of RAM. It takes an average of 2.3 seconds to extract features from 1,000 flows in PCAP. After that, XGBoost can make predictions for the same file in less than 18 milliseconds per flow. This throughput is good for real-time network monitoring, where flow records are sent almost in real time.

V. CONCLUSION & FUTURE WORK

This paper gave a single, useful way to classify encrypted network traffic that is very accurate and doesn't need to decrypt the payload. This keeps users' privacy and cuts down on the amount of computing power needed. By only using statistical flow-level features, the proposed system stays useful even when traditional deep packet

inspection methods stop working because of encryption. The framework works well in a wide range of deployment environments, from edge devices with limited resources that need low-latency inference to cloud-based platforms that can handle large amounts of network traffic.

For the experimental evaluation, we used four different machine learning and deep learning models: Random Forest (RF), XGBoost, Convolutional Neural Networks (CNN), and Long Short-Term Memory (LSTM) networks. Each model had its own strengths that helped with the classification task. The LSTM model was especially good at capturing temporal dependencies in network flows, which makes it a good choice for flow segmentation situations where packet boundaries and timing information are not clear or are noisy. CNN-based models learned spatial patterns from structured flow feature representations and did well in comparison to other models. But XGBoost turned out to be the most balanced model, giving the best balance between accuracy, inference speed, and cost. This makes it perfect for real-time and large-scale deployments.

The suspicious activity detection module made the system much more useful in real life, in addition to traffic classification. Domain-informed confidence thresholding made anomaly recall better than what could be done with a standalone Random Forest classifier. This result shows how well hybrid rule-learning methods work. In these methods, expert knowledge is added to traditional machine learning outputs to lower the number of false negatives in applications that are important for security. The combination of file caching and sequential flow simulation in a Flask-based deployment framework also made it easy to turn research-grade models into real-world network security tools. This deployment architecture shows how machine learning systems can be used to analyze and make decisions about live traffic.

There are a number of promising paths for future work that could make the proposed system even more robust and scalable. First, using Explainable Artificial Intelligence (XAI) techniques like SHAP or feature attribution methods would help network administrators and security analysts understand and trust each classification decision. This is important for compliance and forensic analysis. Second, the system can be made to work with ultra-high-throughput links that are faster than 10 Gbps by horizontally scaling the inference pipeline in the cloud using container orchestration and load balancing techniques. Third, using graph neural network (GNN) architectures to model how multiple flows interact with each other may help find coordinated or distributed multi-flow attacks by finding inter-flow dependencies that single-flow features don't show. Finally, using continuous or online learning methods that change model parameters based on changing traffic patterns would help prevent concept drift and make sure the system works well in the long term, even when training datasets are static.

Overall, the proposed framework lays a strong foundation for intelligent, scalable, and privacy-preserving encrypted traffic analysis. It also connects academic research with network security solutions that can be used in the real world.

ACKNOWLEDGMENT

The authors would like to thank Mr. P. Raja Sekhar, an Assistant Professor in the Department of Computer Science and Engineering at Narayana Engineering College in Nellore, India, for his help. The authors also want to thank the Canadian Institute for Cybersecurity for making the ISCX VPN-NonVPN dataset available to the public for research.

REFERENCES

[1] "Deep Packet: A New Way to Use Deep Learning to Classify Encrypted Traffic," IEEE Research Conference, 2017.

[2] "PacketCGAN: Encrypted Traffic Classification Using Conditional GAN," arXiv preprint, 2019.

[3] "Using Neural Networks to Classify VPN Encrypted Traffic," arXiv preprint, 2020.

“A New Classification Method for Encrypted Internet Traffic Using Machine Learning,” Turkish Journal of Electrical Engineering & Computer Sciences, 2021.

“Classification of Virtual Private Networks Encrypted Traffic Using Ensemble Learning Algorithms,” Egyptian Informatics Journal, 2022.

“A VPN-Encrypted Traffic Identification Method Based on Ensemble Learning,” Applied Sciences (MDPI), 2022.

[7] "ET-BERT: A Contextualized Datagram Representation for Encrypted Traffic Classification," an arXiv preprint from 2022.

[8] "A Deep Learning-Based Encrypted VPN Traffic Classification Method Using Packet Block Image," Electronics (MDPI), 2023.

“IP Packet-Level Encrypted Traffic Classification Using Machine Learning with Lightweight Feature Engineering,” Journal of Information Security and Applications, 2023.

[10] "Research on QoS Classification of Encrypted Traffic Behavior Based on Machine Learning," Electronics (MDPI), 2023.

[11] "Encrypted Network Traffic Classification Based on Machine Learning," Ain Shams Engineering Journal, 2024.

[12] "One Train for Two Tasks: An Encrypted Traffic Classification Framework Using Contrastive Learning," arXiv preprint, 2024.

[13] "A Survey of Encrypted Network Traffic Analysis and Classification Using Machine Learning," Journal of Network and Computer Applications, 2024.

[14] “Machine Learning Approach for Classifying Encrypted Tor Traffic Using XGBoost,” International Journal of Scientific Research in Science, Engineering and Technology, 2025.

[15] "A Comprehensive Review on Machine Learning-Based VPN Detection," Computer Science Review, 2025.